



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе
по дисциплине «Микропроцессорные системы»
на тему:

Устройство для управления вентилятором

Студент

ИУ6-74Б

(Группа)

(Подпись, дата)

Д.О. Кошенков

(И.О. Фамилия)

Руководитель

(Подпись, дата)

Б.И. Бычков

(И.О. Фамилия)

2025 г.

а.м.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой ИУ6

 А.В. Пролетарский

« 2 » сентября 2025 г.

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине Микропроцессорные системы

Студент группы ИУ6-74Б

Кошенков Д.О.

Тема курсовой работы: Устройство для управления вентилятором

Направленность курсовой работы: учебная

Источник тематики (кафедра, предприятие, НИР): кафедра

График выполнения работы: 25% – 4 нед., 50% – 8 нед., 75% – 12 нед., 100% – 16 нед.

Техническое задание:

Разработать на основе микроконтроллера устройство для управления вентилятором.

Обеспечить регулирование работы вентилятора в зависимости от показаний датчика температуры, а также возможность изменения скорости вращения вентилятора. Реализовать двусторонний обмен данными и командами управления по протоколу UART, вывод статуса системы на символьный ЖК-дисплей и сохранение конфигурационных параметров в энергонезависимой памяти EEPROM.

Разработать схему, алгоритмы и программу. Отладить проект в симуляторе или на макете.

Оценить потребляемую мощность. Описать принципы и технологию программирования используемого микроконтроллера.

Оформление курсовой работы:

1. Расчетно-пояснительная записка на 30-35 листах формата А4.

2. Перечень графического материала:

- а) схема электрическая функциональная;
- б) схема электрическая принципиальная.

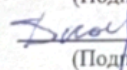
Дата выдачи задания: «2» сентября 2025 г.

Руководитель курсовой работы

Студент

 02.09.2025
(Подпись, дата)

Б.И. Бычков
(И.О.Фамилия)

 02.09.2025
(Подпись, дата)

Д.О. Кошенков
(И.О.Фамилия)

Примечание: Задание оформляется в двух экземплярах; один выдается студенту, второй хранится на кафедре.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Конструкторская часть	6
1.1 Анализ требований технического задания	6
1.2 Разработка структурной схемы	6
1.3 Обоснование выбора элементной базы	7
1.3.1 Выбор микроконтроллера	7
1.3.2 Выбор датчика температуры	8
1.4 Выбор двигателя и драйвера	9
1.4.1 Выбор средств индикации	9
1.4.2 Выбор преобразователя интерфейса	10
1.5 Разработка функциональной схемы	11
1.6 Описание микроконтроллера ATmega8515	12
1.7 Принцип работы DRV8871	13
1.8 Описание принципиальной электрической схемы	14
1.9 Анализ временных характеристик интерфейсов	17
1.9.1 Протокол 1-Wire датчика DHT11	17
1.9.2 Интерфейс UART (асинхронная последовательная передача)	18
1.9.3 Интерфейс жидкокристаллического дисплея LM016L	19
1.10 Структура данных, передаваемых датчиком	21
1.11 Описание программной реализации и алгоритмов	22
1.11.1 Алгоритм основного цикла	22
1.11.2 Алгоритм обработки обновления состояния вентилятора	23
1.11.3 Алгоритм обработки команды UART	24
1.12 Питание устройства	25
1.13 Интерфейс программирования	26
1.14 Расчетная часть	27
1.14.1 Расчет элементов драйвера двигателя	27
1.14.2 Энергетический расчет и выбор источника питания	27
1.14.3 Расчет компонентов периферийных узлов	28
1.14.4 Хранение настроек в энергонезависимой памяти EEPROM	29

1.14.5 Методы программирования микроконтроллера	30
2 Технологическая часть	31
2.1 Работа в системах разработки и отладки	31
2.2 Структура программного обеспечения	32
2.3 Настройка периферийных устройств	33
2.4 Отладка и тестирование	33
ЗАКЛЮЧЕНИЕ	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
ПРИЛОЖЕНИЕ А ФУНКЦИОНАЛЬНАЯ ЭЛЕКТРИЧЕСКАЯ СХЕМА	37
ПРИЛОЖЕНИЕ Б ФУНКЦИОНАЛЬНАЯ ЭЛЕКТРИЧЕСКАЯ СХЕМА	39
ПРИЛОЖЕНИЕ В ИСХОДНЫЙ ТЕКСТ ПРОГРАММЫ	43

ВВЕДЕНИЕ

Системы активного охлаждения являются неотъемлемой частью современной электроники, обеспечивая отвод тепла от греющихся компонентов. Отсутствие интеллектуального управления приводит к повышенному шуму, износу вентиляторов и нерациональному расходу электроэнергии.

Целью данной работы является разработка автоматизированной системы управления вентилятором на базе 8-битного микроконтроллера, способной работать как автономно, так и под внешним управлением.

Для достижения цели решены следующие задачи: – обоснован выбор элементной базы (МК ATmega8515, датчик DHT11, драйвер DRV8871);

- разработана схема электрическая принципиальная;
- реализованы алгоритмы считывания данных по нестандартному протоколу, генерации ШИМ и работы с энергонезависимой памятью;
- выполнена программная реализация устройства на языке C.

1 Конструкторская часть

1.1 Анализ требований технического задания

Согласно техническому заданию, необходимо разработать на основе микроконтроллера устройство автоматизированного управления системой активного охлаждения.

Устройство должно обеспечивать регулирование работы вентилятора в зависимости от текущих показаний датчика температуры. Также устройство должно предоставлять возможность ручного управления скоростью вращения вентилятора посредством ШИМ.

Важным требованием является реализация двустороннего обмена данными и командами управления по протоколу UART. Микроконтроллер должен принимать команды настройки, например, изменение температурных порогов или скорости и передавать информацию о статусе системы внешнему устройству. Для отображения этой информации непосредственно на устройстве необходимо использовать символьный жидкокристаллический дисплей.

Для обеспечения сохранения пользовательских настроек при отключении питания требуется использование энергонезависимой памяти EEPROM.

Поскольку вентилятор является индуктивной нагрузкой и потребляет ток, значительно превышающий нагрузочную способность выводов микроконтроллера, необходимо использовать силовой драйвер двигателя. Драйвер должен обеспечивать согласование уровней напряжения и тока, а также поддерживать управление скоростью через ШИМ.

Для подключения к персональному компьютеру, который зачастую не имеет нативного COM-порта, устройство целесообразно снабдить преобразователем интерфейсов USB-UART или соответствующим разъемом для подключения внешнего адаптера.

1.2 Разработка структурной схемы

Структурная схема устройства разработана на основе анализа технического задания и определяет основные функциональные узлы системы. Согласно требованиям, устройство состоит из следующих блоков:

- МК — центральное устройство управления, осуществляющее обработку данных и формирование управляющих сигналов;
- датчик — цифровой модуль измерения температуры и влажности;
- мотор — исполнительный механизм (вентилятор) системы охлаждения;
- дисплей — средство визуального отображения текущих параметров системы;
- кнопки — интерфейс пользователя для ручной настройки параметров;
- разъем подключения к ПК — обеспечивает физическое соединение для обмена данными.

На рисунке 1 представлена структурная схема устройства.

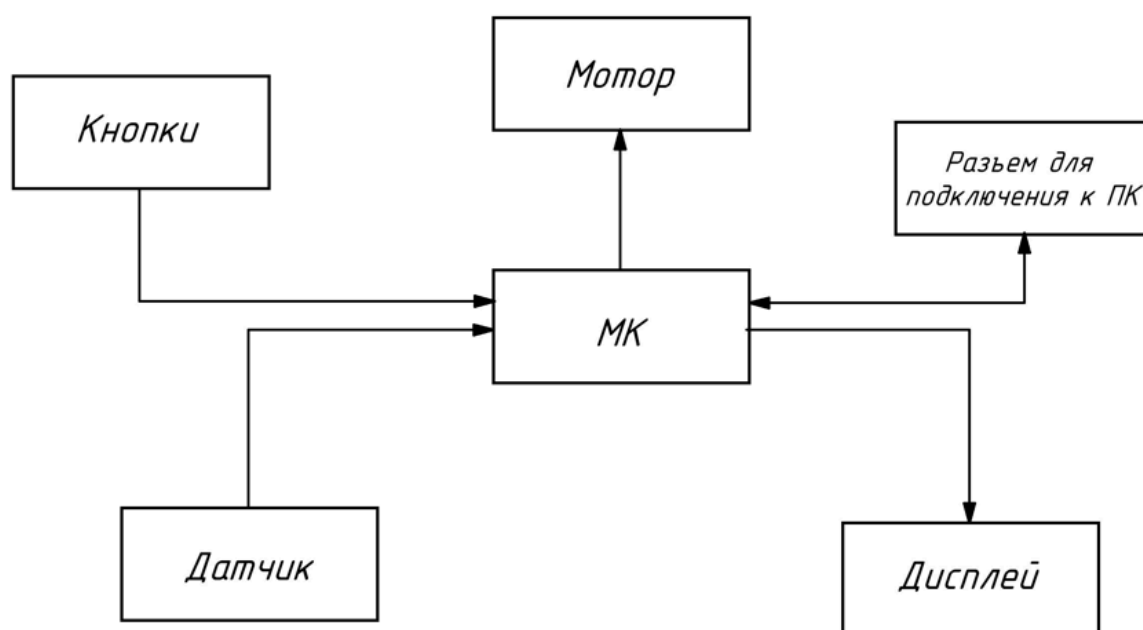


Рисунок 1 — Структурная схема

1.3 Обоснование выбора элементной базы

Выбор компонентов производился на основе анализа технических требований, сравнительных характеристик аналогов, доступности и удобства монтажа.

1.3.1 Выбор микроконтроллера

В качестве управляющего устройства рассматривались 8-битные микроконтроллеры архитектуры AVR. Основными критериями выбора являлись: объем памяти программ Flash и данных EEPROM, количество портов

ввода-вывода GPIO и наличие необходимых аппаратных интерфейсов: UART, Timer/Counter. Сравнительный анализ представлен в таблице 1.

Таблица 1 — Сравнительные характеристики микроконтроллеров

Параметр	ATmega8515	ATmega8	ATmega328P
Flash-память, КБ	8	8	32
EEPROM, байт	512	512	1024
SRAM, байт	512	1024	2048
Линии ввода-вывода	35	23	23
Таймеры (8/16 бит)	1 / 1	2 / 1	2 / 1
Корпус	DIP-40	DIP-28	DIP-28

Для реализации проекта выбран микроконтроллер ATmega8515 [1]. Несмотря на меньший объем SRAM по сравнению с аналогами, его ресурсов достаточно для решения поставленной задачи. Наличие 512 байт EEPROM удовлетворяет требованиям по хранению настроек.

1.3.2 Выбор датчика температуры

Для мониторинга параметров окружающей среды рассматривались цифровые и аналоговые датчики. Сравнение приведено в таблице 2.

Таблица 2 — Сравнительные характеристики датчиков

Параметр	DHT11	DS18B20	LM35
Тип	Цифровой	Цифровой	Аналоговый
Измеряемые величины	T + H	T	T
Диапазон T, °C	0...50	-55...+125	-55...+150
Точность T, °C	±2	±0.5	±0.5
Интерфейс	1-Wire	1-Wire	АЦП

Выбран датчик DHT11 [2]. Датчик предназначен для использования в жилых помещениях, где диапазон 0...50 °C является достаточным.

Использование цифрового интерфейса исключает необходимость калибровки АЦП и снижает влияние помех на линии связи.

1.4 Выбор двигателя и драйвера

В системе используется коллекторный двигатель постоянного тока с напряжением 12 вольт. Такое решение безопаснее, чем использование вентиляторов с питанием от бытовой сети 220 вольт. По сравнению с шаговыми двигателями выбранный вентилятор создает значительно меньше шума во время работы, что важно для домашнего устройства.

Для управления двигателем постоянного тока необходим драйвер, обеспечивающий усиление по току и возможность регулирования скорости. Сравнение вариантов приведено в таблице 3.

Таблица 3 — Сравнительные характеристики драйверов

Параметр	DRV8871	L298N	Дискр. MOSFET
Тип	Н-мост	Н-мост	Ключ
Макс. ток, А	3.6	2.0	>10
Напряжение, В	6.5...45	до 46	Зависит от ти- па
Защита	ОСР, TSD, UVLO	Нет	Внешняя
КПД	Высокий	Низкий	Высокий

Для управления двигателем применяется микросхема DRV8871 [3]. Она предпочтительнее старых моделей драйверов, таких как L298N, поскольку практически не нагревается в процессе работы и не требует установки радиатора. Также этот драйвер имеет встроенную защиту от короткого замыкания и перегрева, что делает итоговое устройство компактным и надежным без усложнения электрической схемы.

1.4.1 Выбор средств индикации

Для отображения информации пользователю рассматривались различные типы дисплеев.

Таблица 4 — Сравнение средств индикации

Параметр	LCD 1602 (LM016L)	OLED 0.96"
Тип информации	Символьный	Графический
Интерфейс	Параллельный (4/8 бит)	I2C/SPI
Читаемость	Высокая	Высокая
Потребление	Среднее	Низкое

Выбран жидкокристаллический модуль LM016L [4]. Он предназначен для вывода текстовой информации и простых символов. Главной особенностью данного модуля является универсальный параллельный интерфейс, который поддерживает передачу данных как по восьмибитной, так и по четырехбитной шине. Это позволяет адаптировать схему подключения под имеющееся количество свободных выводов микроконтроллера, сохраняя при этом простоту программной реализации протокола обмена.

1.4.2 Выбор преобразователя интерфейса

Для подключения микроконтроллера (UART TTL) к порту USB персонального компьютера необходим преобразователь интерфейсов.

Таблица 5 — Сравнение преобразователей USB-UART

Параметр	CH340N	FT232RL	CP2102
Корпус	SOP-8	SSOP-28	QFN-28
Обвязка	Минимальная	Средняя	Средняя
Встроенный генератор	Да	Да	Да

Выбрана микросхема CH340N [5]. Она выпускается в компактном корпусе SOP-8 и требует минимального количества внешних компонентов, так

как имеет встроенный тактовый генератор. Это упрощает разводку печатной платы и снижает стоимость устройства.

1.5 Разработка функциональной схемы

Функциональная схема устройства разработана на основе структурной схемы и выбранной элементной базы. Она детализирует внутреннюю архитектуру микроконтроллера, распределение его логических ресурсов и организацию связей с периферийными модулями. Схема электрическая функциональная приведена в приложении А.

Ядром системы является микроконтроллер ATmega8515, который на схеме представлен в виде совокупности функциональных блоков: арифметико-логического устройства, подсистемы памяти, включающей Flash-память программ, оперативное запоминающее устройство и энергонезависимую память EEPROM. Взаимодействие с внешним миром и внутренними процессами обеспечивают периферийные модули, такие как таймеры-счетчики, контроллер прерываний, последовательные интерфейсы SPI и USART, а также порты ввода-вывода.

Синхронизация работы вычислительного ядра осуществляется системой тактирования, построенной на базе внешнего кварцевого резонатора и внутреннего генератора микроконтроллера. Такое решение гарантирует высокую стабильность частоты, необходимую для корректной передачи данных по временным протоколам.

Блок пользовательского ввода состоит из группы коммутационных элементов, подключенных к линиям порта А. Сигналы от кнопок дополнительно объединены логической схемой для формирования запроса внешнего прерывания. Это позволяет микроконтроллеру мгновенно реагировать на действия оператора, выходя из режима ожидания или прерывая текущую задачу для обработки команды управления.

Система визуализации реализована на символьном жидкокристаллическом дисплее. Обмен информацией с индикатором происходит через порт С, где часть линий используется для параллельной передачи данных, а отдельные

линии выделены для управляющих сигналов синхронизации и выбора режима работы дисплея.

Исполнительный узел управления вентилятором включает специализированный драйвер двигателя, согласующий логические уровни микроконтроллера с силовой частью. Управление осуществляется через выходные линии порта, одна из которых задает режим работы драйвера, а вторая, подключенная к выходу таймера-счетчика, формирует сигнал широтно-импульсной модуляции для плавного регулирования оборотов двигателя.

Мониторинг параметров окружающей среды выполняется цифровым датчиком температуры и влажности. Обмен данными с микроконтроллером производится по однопроводному двунаправленному интерфейсу, подключенному к линии порта D, что позволяет считывать показания в цифровом виде без использования аналого-цифрового преобразования.

Канал связи с персональным компьютером организован на базе преобразователя интерфейсов USB-UART. Данный узел обеспечивает трансляцию сигналов шины USB в последовательный асинхронный протокол USART микроконтроллера, позволяя осуществлять дистанционный мониторинг и настройку системы.

1.6 Описание микроконтроллера ATmega8515

В качестве вычислительного ядра проектируемой системы выбран 8-битный микроконтроллер ATmega8515, построенный на базе архитектуры AVR RISC. Данная архитектура характеризуется разделением шин доступа к памяти программ и памяти данных, что обеспечивает возможность одновременной выборки инструкции и данных. Большинство команд выполняется за один машинный цикл, благодаря чему устройство достигает высокой производительности при тактовой частоте 16 МГц [1]. Структурная организация включает 32 рабочих регистра общего назначения, непосредственно связанных с арифметико-логическим устройством, что позволяет выполнять операции за один такт.

Подсистема памяти микроконтроллера разделена на три независимых области. Первая представляет собой Flash-память объемом 8 Кбайт для

хранения программного кода. Вторая — это оперативная память SRAM объемом 512 байт, используемая для переменных и стека. Третья область — энергонезависимая память EEPROM объемом 512 байт. В рамках проекта она используется для хранения настроек системы, таких как температурные пороги и последний активный режим работы, обеспечивая их сохранность при отключении питания.

Для реализации функций управления задействованы основные периферийные ресурсы микроконтроллера. Восемьбитный таймер-счетчик сконфигурирован для работы в режиме генерации широтно-импульсной модуляции, что позволяет плавно регулировать мощность электродвигателя. Модуль универсального асинхронного приемопередатчика USART обеспечивает двусторонний обмен данными с компьютером для мониторинга и отладки. Система внешних прерываний INT0 используется для обработки сигналов от кнопок управления, позволяя мгновенно реагировать на действия пользователя. Порты ввода-вывода общего назначения программно управляют параллельным интерфейсом дисплея, обменом данными с цифровым датчиком DHT11 и логическими уровнями драйвера двигателя.

1.7 Принцип работы DRV8871

Необходимость использования специализированного драйвера обусловлена несоответствием энергетических характеристик управляющего контроллера и исполнительного механизма. Порты ввода-вывода микроконтроллера ATmega8515 рассчитаны на нагрузку не более 20 мА, тогда как номинальный ток потребления выбранного вентилятора составляет 300 мА. Прямое подключение нагрузки вызвало бы перегрузку выходных каскадов микроконтроллера, поэтому для коммутации силовой цепи применяется интегральная микросхема DRV8871.

Данный драйвер способен обеспечивать пиковый выходной ток до 3.6 А [3]. Сопоставление этого параметра с рабочим током вентилятора (0.3 А) показывает наличие более чем десятикратного запаса по мощности. Это гарантирует надежную работу системы даже при пусковых перегрузках или возможном подклинивании вала двигателя.

Функциональная схема драйвера, отображающая внутренние связи между логическим блоком и силовой частью, приведена на рисунке 2.

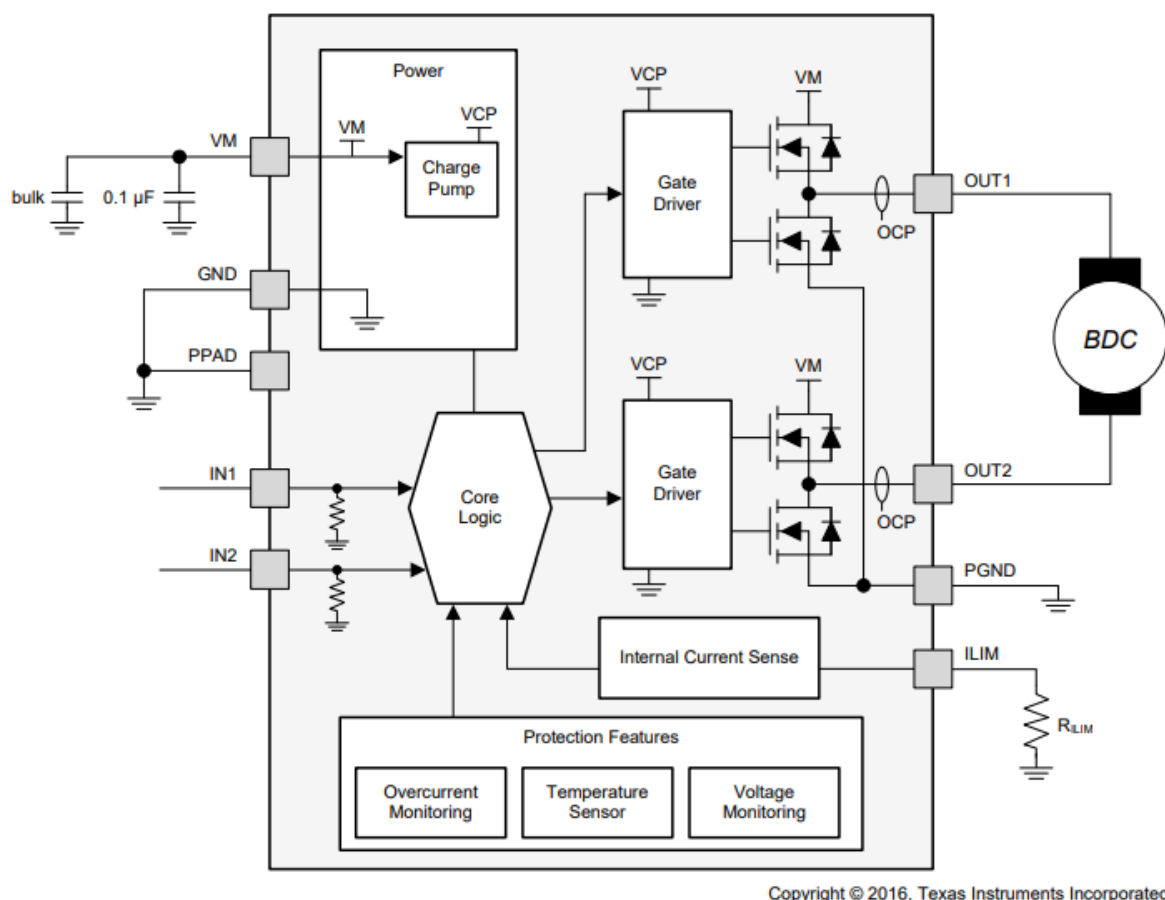


Рисунок 2 — Функциональная схема драйвера DRV8871

Логика управления драйвером реализована через два входа IN1 и IN2. Согласно таблице истинности, приведенной в технической документации, комбинация логических уровней на этих входах определяет состояние выходов OUT1 и OUT2: вращение вперед, вращение назад, торможение или свободный выбег. Входы оснащены внутренними стягивающими резисторами, что предотвращает несанкционированное включение двигателя при высокоимпедансном состоянии выводов управляющего микроконтроллера.

Для реализации функции ограничения тока драйвер использует схему измерения падения напряжения на транзисторах. Порог срабатывания защиты задается единственным внешним резистором на выводе ILIM, что исключает необходимость применения мощных токоизмерительных шунтов в цепи двигателя.

1.8 Описание принципиальной электрической схемы

Принципиальная электрическая схема устройства разработана на основании функциональной схемы и определяет полный состав электронных компонентов и связей между ними. Схема изображена в приложении Б.

Центральным элементом системы является микроконтроллер DD4 ATmega8515. Для обеспечения тактирования используется внешний кварцевый резонатор Z1, подключенный к выводам XTAL1 и XTAL2. Цепь начального сброса реализована на резисторе R7 (10 кОм), который подтягивает вывод RESET к шине питания, предотвращая ложные срабатывания. Кнопка SA1, подключенная к выводу сброса через конденсатор, предназначена для ручного сброса микроконтроллера пользователем. Разъем XP3 предназначен для внутрисхемного программирования микроконтроллера.

Система питания устройства построена по импульсной схеме понижения напряжения. Входное напряжение +12 В поступает через разъем «Контакт 1». Для питания силовой части (драйвера двигателя) напряжение +12 В используется напрямую. Для питания логической части применена микросхема DD3 MP2307 — синхронный понижающий преобразователь напряжения. Резистивный делитель R5-R4 в цепи обратной связи, вывод FB, задает выходное напряжение на уровне 5 В. Резистор R2 подтягивает вход включения к питанию, обеспечивая автоматический запуск преобразователя.

Управление электродвигателем М осуществляется через специализированный драйвер DD5 DRV8871. Микросхема подключена к силовой шине +12 В, вывод VM. Конденсаторы C10 22 пФ и C11 100 нФ выполняют функцию фильтрации питания драйвера, предотвращая просадки напряжения при коммутации обмоток двигателя. Резистор R6 30 кОм, подключенный к выводу ILIM, задает порог ограничения тока двигателя на уровне 2.1 А, защищая его и источник питания от перегрузок. Управляющие сигналы поступают от микроконтроллера на входы IN1 и IN2. Выходы OUT1 и OUT2 подключены непосредственно к клеммам двигателя постоянного тока.

Блок индикации реализован на жидкокристаллическом дисплее DD6 LM016L. Подключение выполнено по четырехбитной шине данных, D4-D7 дисплея подключены к выводам PC4-PC7 микроконтроллера. Управляющие

сигналы RS и E подключены к выводам PC2 и PC3 соответственно. Вывод R/W дисплея соединен с землей, что переводит индикатор в режим постоянной записи. Подстроечный резистор R8 10 кОм образует делитель напряжения для регулировки контрастности изображения на дисплее.

Интерфейс ввода включает четыре тактовые кнопки SA2-SA5, подключенные к порту A микроконтроллера (PA0-PA3). Схемотехническое решение включает диодную развязку на элементах VD1-VD4. Аноды диодов подключены к линиям кнопок, а катоды объединены и заведены на линию внешнего прерывания PD2/INT0. Такое включение позволяет микроконтроллеру фиксировать факт нажатия любой из кнопок через генерацию прерывания, выводя устройство из режима энергосбережения, после чего программа определяет конкретную нажатую кнопку путем опроса порта A.

Взаимодействие с персональным компьютером организовано через интерфейс USB. Преобразование интерфейса USB в UART выполняет микросхема DD2 (CH340N). Линии D+ и D- подключены непосредственно к разъему USB. Линии приема (RXD) и передачи (TXD) соединены с соответствующими выводами порта D микроконтроллера (PD0 и PD1). Конденсатор C12 (100 нФ) служит для фильтрации внутреннего опорного напряжения микросхемы CH340N.

Датчик температуры и влажности DD1 (DHT11) подключен к шине питания +5 В. Информационный вывод датчика соединен с портом ввода-вывода микроконтроллера (PD7). Конденсатор C1 (22 пФ) подключен между выводом питания датчика и землей для стабилизации питания и снижения помех. Резистор R1 (5.1 кОм) обеспечивает подтяжку информационной линии датчика к высокому уровню, что является требованием спецификации протокола 1-Wire.

Представленная принципиальная схема обеспечивает реализацию всех функций, заложенных в техническом задании, включая автоматическое и ручное управление, индикацию, защиту силовых цепей и обмен данными с внешними устройствами.

1.9 Анализ временных характеристик интерфейсов

1.9.1 Протокол 1-Wire датчика DHT11

Взаимодействие между микроконтроллером и датчиком температуры-влажности DHT11 реализуется по однопроводному интерфейсу с открытым коллектором. Шина подтягивается резистором R1 номиналом 5.1 кОм к питанию +5 В.

Микроконтроллер инициирует цикл обмена путем удержания шины в состоянии LOW не менее 18 мс, затем отпускает её. Датчик обнаруживает переход сигнала и переходит из режима сна в рабочий режим. После этого датчик формирует ответный импульс: удерживает линию LOW в течение 40 мкс, затем отпускает на 40 мкс, подтверждая готовность к передаче данных.

Далее датчик передает 40 бит полезной информации. Каждый бит кодируется длительностью LOW-импульса на шине: логический ноль занимает примерно 26 мкс, единица — около 70 мкс. Между битами шина находится в состоянии HIGH на протяжении 54 мкс, что позволяет микроконтроллеру измерить длительность предыдущего импульса.

Содержимое 40-битного пакета распределено следующим образом: первые 8 бит — целая часть влажности в процентах, следующие 8 бит — дробная часть влажности, затем 8 бит целой части температуры в градусах Цельсия, 8 бит дробной части, и наконец 8 бит контрольной суммы для верификации.

Расчет полного времени цикла опроса: стартовый сигнал занимает 18 мс, ответный импульс — 0.08 мс, передача 40 бит при среднем значении (смешанные нули и единицы) — примерно 1.6 мс. Итого получается около 19.7 мс.

$$t_{\text{cycle}} = 18 + 0.08 + 40 \times \frac{26 + 70}{2} \times 10^{-3} = 19.7 \text{ мс} \quad (1)$$

Опрос датчика в прошивке выполняется не чаще одного раза в 2 секунды. Такой интервал превышает минимально допустимый по спецификации (1

секунда), что обеспечивает защиту от перегрева кристалла датчика и улучшает надежность измерений.

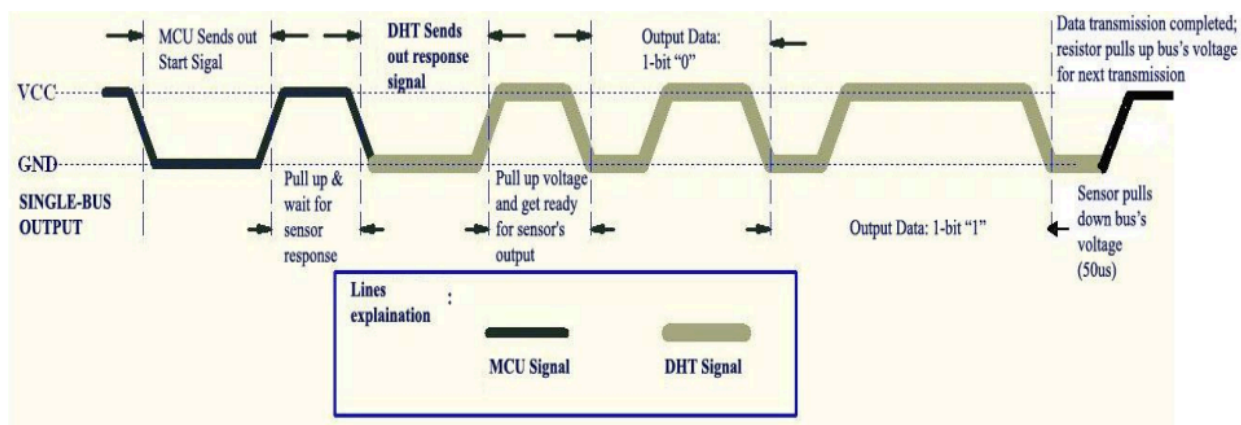


Рисунок 3 — Временная диаграмма протокола 1-Wire DHT11

1.9.2 Интерфейс UART (асинхронная последовательная передача)

Последовательный интерфейс UART на микроконтроллере ATmega8515 организует двусторонний обмен данными с персональным компьютером. Связь проходит через микросхему-преобразователь CH340N, которая транслирует протокол USB в стандартный UART. Линия приема RXD подключена к выводу PD0, линия передачи TXD — к выводу PD1.

Стандартный формат кадра UART состоит из десяти временных интервалов (битов): один стартовый бит LOW, восемь бит данных (младший разряд передается первым) и один стоп-бит HIGH.

Установленная скорость обмена составляет 9600 бит в секунду, что соответствует периоду одного бита в 104.17 микросекунд. Регистр делителя частоты UBRR микроконтроллера рассчитывается по формуле для асинхронного режима:
$$UBRR = \frac{f_{cpu}}{16 \times BAUD} - 1 = \frac{8 \times 10^6}{153600} - 1 = 51$$

При таком значении UBRR реальная скорость передачи составляет 9615.38 бит/с, что отличается от номинальной на 0.16 процента. Такая ошибка находится в пределах допустимого диапазона ± 3 процента, обеспечивая надежную синхронизацию между передатчиком и приемником.

Полная длительность одного кадра равна 1041.7 микросекунды, или примерно 1 миллисекунда. При отправке команды управления из пяти символов (например, значение напряжения из трех цифр и символ перевода строки) общее время передачи составит около 5.2 миллисекунды.

Микроконтроллер обрабатывает входящие данные через прерывающий обработчик, который срабатывает при получении каждого байта. Входящие символы накапливаются в кольцевом буфере емкостью 32 байта. Когда приемник обнаруживает символ перевода строки, устанавливается внутренний флаг готовности команды. Основной цикл программы проверяет этот флаг и обрабатывает полученную команду. Время от последнего полученного символа до установки флага не превышает одного машинного цикла, то есть менее 125 наносекунд.

Максимальное время ответа устройства на команду с ПК складывается из времени передачи команды (несколько миллисекунд), времени обработки в основном цикле (не более 100 микросекунд) и времени передачи ответного кадра (от 10 до 20 миллисекунд). Типичное время ответа не превышает 30 миллисекунд.



Рисунок 4 — Временная диаграмма формата кадра UART

1.9.3 Интерфейс жидкокристаллического дисплея LM016L

Дисплей на 16 символов в две строки подключен в режиме 4-битной шины данных. Линии D4, D5, D6, D7 дисплея соединены с выводами PC4, PC5, PC6, PC7 микроконтроллера соответственно. Управляющие сигналы регистра выбора (RS) и сигнал синхронизации (E) подключены к выводам PC2 и PC3. Линия чтения-записи (R/W) заземлена, что переводит дисплей в режим только записи.

При четырехбитном режиме передачи каждый байт данных передается двумя отдельными операциями: сначала четыре старших бита, затем четыре младших. Каждая половина байта сопровождается импульсом синхронизации на линии E.

Последовательность записи одного символа выглядит следующим образом: на линиях данных устанавливаются четыре старших бита, после чего формируется импульс сигнала синхронизации E (переход от HIGH к LOW и обратно в HIGH), затем на тех же линиях устанавливаются четыре младших бита и вновь создается импульс E. Такой порядок позволяет дисплею правильно интерпретировать входящие данные.

Согласно технической документации дисплея, данные должны установиться на линиях как минимум за 40 наносекунд до переднего фронта импульса синхронизации, импульс синхронизации должен иметь длительность не менее 230 наносекунд, а данные должны оставаться стабильными как минимум 10 наносекунд после окончания импульса.

При реализации в прошивке каждая операция записи полубайта требует около 1 микросекунды, что обеспечивает достаточный запас по всем временным параметрам. Полный байт записывается за 2 микросекунды. Инициализация дисплея при включении устройства занимает около 32 микросекунды и включает установку режима работы, включение подсветки и определение начальной позиции курсора.

Обновление всего видимого содержимого дисплея (32 символа на две строки плюс служебные команды позиционирования) требует примерно 100 микросекунд. В основной программе буфер дисплея обновляется с частотой 10 Гц, то есть один раз в 100 миллисекунд. Такая частота обновления достаточна для восприятия человеческим глазом и не создает заметного мерцания изображения.

Реализованы две основные функции для работы с дисплеем: функция позиционирования курсора позволяет выбрать координаты (строку и позицию в строке), а функция вывода строки символов позволяет записать текст в выбранную позицию. Обе операции буферизованы, что означает, что микроконтроллер не занят другими задачами во время обновления дисплея.

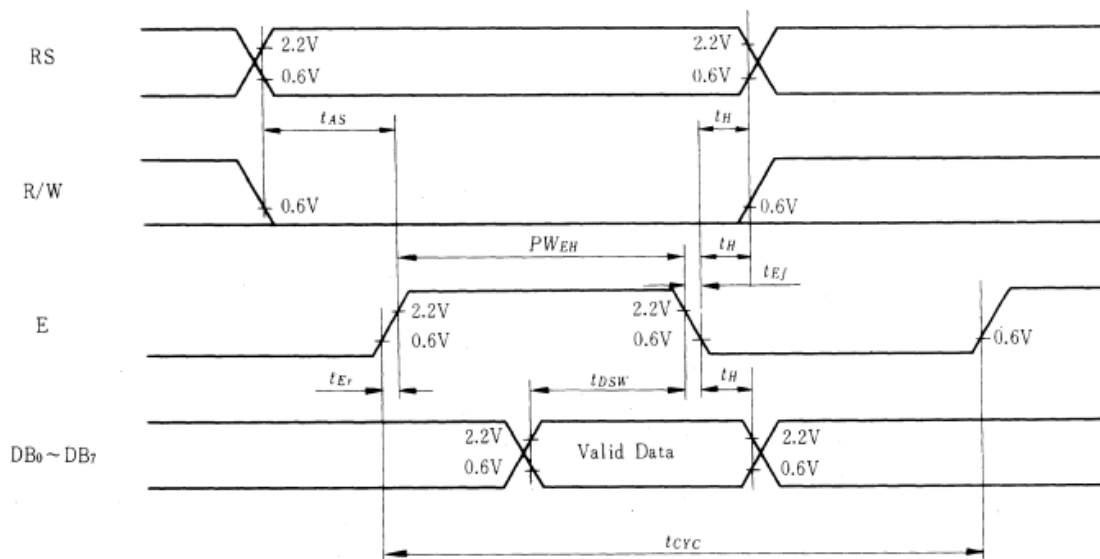


Рисунок 5 — Временная диаграмма интерфейса LM016L

1.10 Структура данных, передаваемых датчиком

После завершения стартовой последовательности датчик DHT11 передает ровно 40 бит последовательно в один момент времени. Эти 40 бит организованы в пять байт информации, каждый из которых несет определенный вид данных.

Первый байт содержит целую часть влажности в процентах в диапазоне от 20 до 95 процентов. Второй байт (биты 8–15) содержит дробную часть влажности, которая в датчике DHT11 редко используется и часто равна нулю.

Третий байт содержит целую часть температуры в градусах Цельсия со знаком. Датчик измеряет температуру в диапазоне от -40 до $+125$ градусов. Четвертый байт содержит дробную часть температуры, также редко используемую в практических приложениях.

Пятый байт является контрольной суммой и равен сумме всех четырех предыдущих байт по модулю 256, что позволяет микроконтроллеру проверить целостность полученных данных.

$$\text{Пакет} = [H_{\text{целая}}] [H_{\text{дробная}}] [T_{\text{целая}}] [T_{\text{дробная}}] [\text{CRC}] \quad (2)$$

где H — влажность в процентах, T — температура в градусах Цельсия, CRC — контрольная сумма.

1.11 Описание программной реализации и алгоритмов

1.11.1 Алгоритм основного цикла

После подачи питания происходит инициализация портов ввода-вывода, настройка UART, LCD-дисплея и внешних прерываний. Далее из энергонезависимой памяти EEPROM загружаются сохраненные настройки: режим работы, пороги температуры и скорость вентилятора.

В основном цикле последовательно выполняются следующие действия:

1) проверяется флаг готовности команды `cmd_ready`, и в случае его установки происходит разбор принятой строки с обновлением глобальных переменных и сохранением данных в EEPROM;

2) с помощью программного счетчика и функции задержки формируется интервал опроса датчика, по истечении которого вызывается функция чтения данных;

3) в автоматическом режиме устройство сравнивает текущую температуру с заданными порогами, включая вентилятор при превышении верхнего порога и выключая его при падении ниже нижнего;

4) при изменении любых параметров или успешном чтении датчика выставляется флаг необходимости обновления дисплея, инициирующий вывод актуальной информации.

Схема алгоритма основного цикла представлена на рисунке 6:

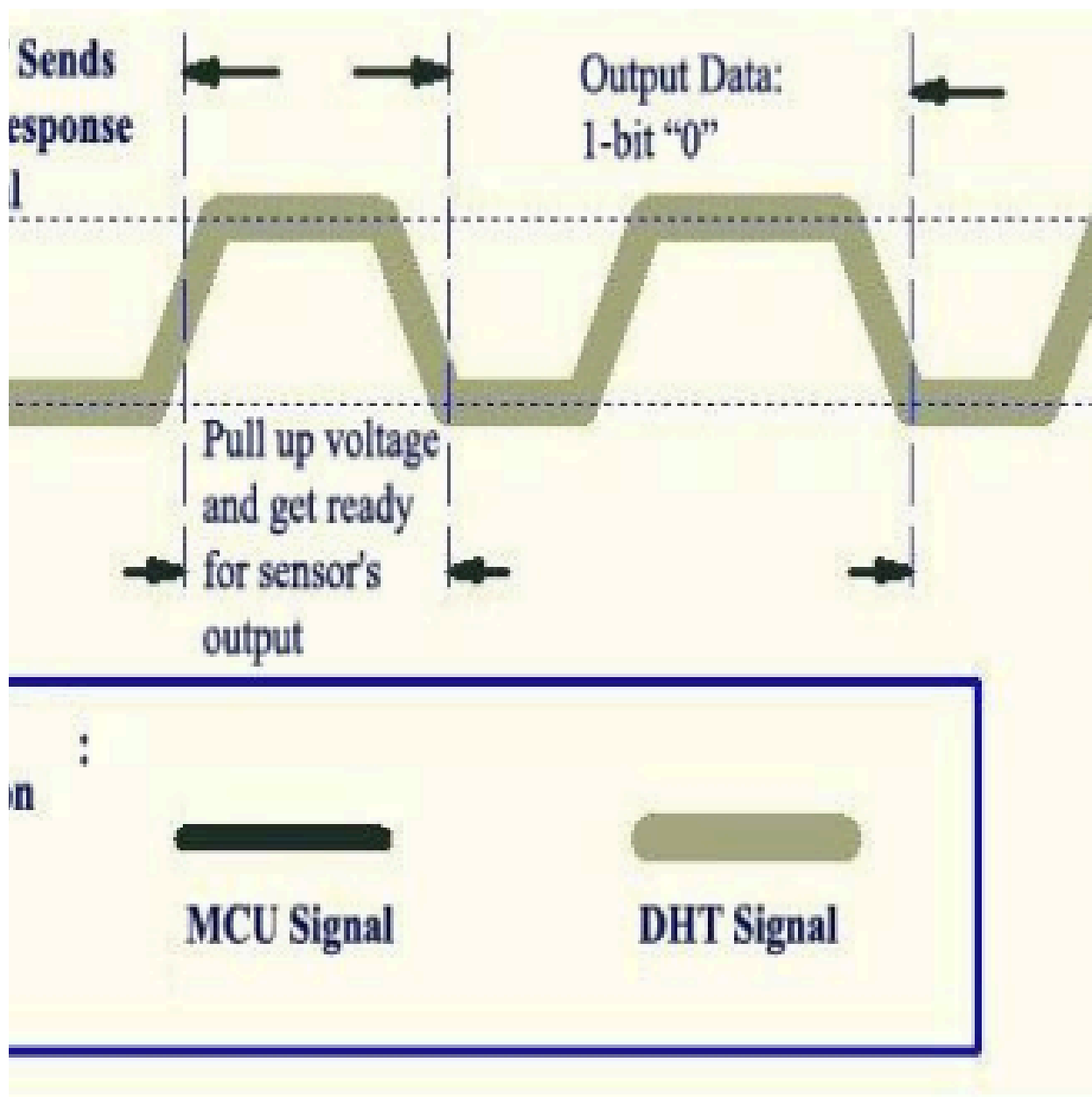


Рисунок 6 — Схема алгоритма основного цикла

1.11.2 Алгоритм обработки обновления состояния вентилятора

Для автоматического управления охлаждением реализован алгоритм, предотвращающий частые переключения двигателя при колебаниях температуры около порогового значения (см. рисунок 7).

Логика работы включает следующие этапы:

- 1) сравнивается текущее значение температуры с заданным верхним порогом, и, если температура превышает его при выключенном вентиляторе, производится включение двигателя;
- 2) в случае нахождения температуры ниже верхнего порога проверяется

условие нижнего порога;

3) если температура опустилась ниже заданного минимума и вентилятор находится в активном состоянии, формируется команда на его выключение, а в остальных случаях состояние системы остается без изменений.



Рисунок 7 — Схема алгоритма обработки обновления состояния вентилятора

1.11.3 Алгоритм обработки команды UART

Обработка входящих данных производится по факту установки флага готовности команды, после чего анализируется содержимое приемного буфера.

Процедура разбора команд выполняется следующим образом:

- 1) проверяется первый символ буфера, определяющий тип запрашиваемого действия;
- 2) для команд ручного управления („1“ и „0“) система переводится в ручной режим, изменяется состояние мотора и обновляется запись в EEPROM;
- 3) при получении команд смены режима („A“ и „M“) устанавливается соответствующий флаг автоматического или ручного управления с сохранением настройки;
- 4) для команд с параметрами („V“, „H“, „L“) производится преобразование строкового аргумента в число, проверка на вхождение в допустимый диапазон (0–255 для скорости, 0–99 для температуры) и обновление соответствующих глобальных переменных и ячеек памяти;
- 5) по завершении операций флаг готовности команды сбрасывается для разрешения приема следующих данных.

Схема алгоритма обработки команды UART представлена на рисунке 8:

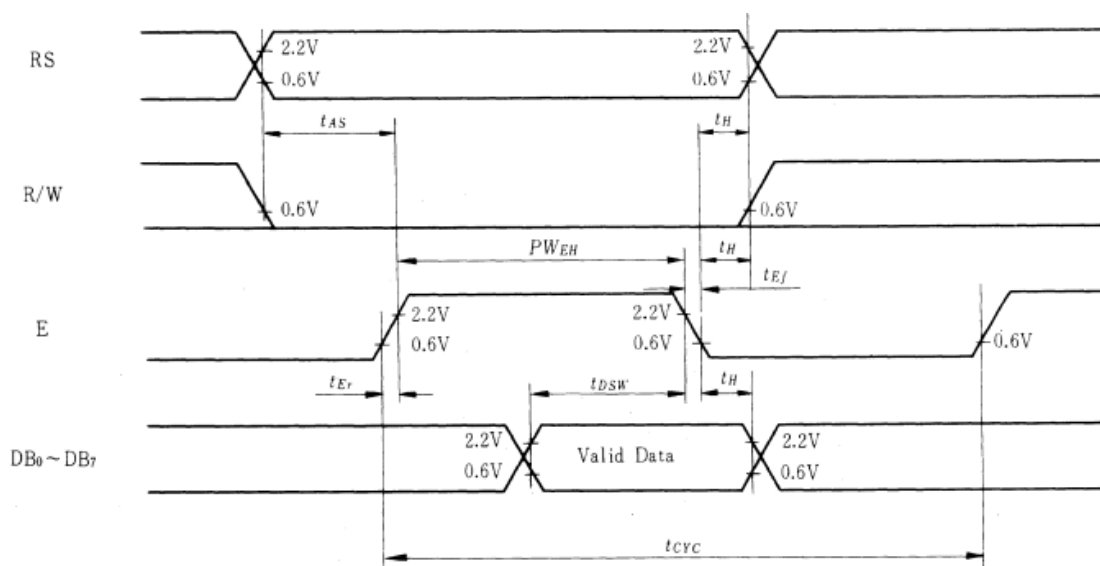


Рисунок 8 — Схема алгоритма обработки команды UART

1.12 Питание устройства

Для обеспечения функционирования устройства выбрана схема с единым входным напряжением +12 В, которое подается через разъем «Контакт 1». Данное напряжение необходимо для питания мощной нагрузки — электродвигателя вентилятора. Однако микроконтроллер, дисплей, датчики и микросхема интерфейса требуют напряжения питания +5 В.

Для преобразования напряжения +12 В в +5 В применена микросхема DD3 — MP2307 [6]. Это монолитный синхронный понижающий импульсный стабилизатор напряжения. Выбор импульсного преобразователя вместо линейного обусловлен необходимостью минимизации тепловых потерь. При понижении 12 В до 5 В линейный стабилизатор рассеивал бы значительную мощность в виде тепла, что потребовало бы установки громоздкого радиатора. КПД микросхемы MP2307 достигает 95%, что обеспечивает высокую энергоэффективность устройства.

Схема включения MP2307 включает:

- входные конденсаторы C7 и C8 22 100нФ, объемный накопитель для фильтрации входного напряжения и стабилизации питания при переходных процессах;
- резистор R2, подтягивающий вход включения к питанию, обеспечивающий автоматический запуск преобразователя при подаче питания;
- дроссель индуктивностью 10 мкГн и выходной конденсатор C5 22 мкФ, образующие LC-фильтр для сглаживания пульсаций выходного напряжения;
- делитель напряжения R5-R4, 43 кОм и 10 кОм соответственно в цепи обратной связи, задающий выходное напряжение стабилизатора на уровне 5.0 В;
- конденсатор C6 100 нФ между выводом COMP и GND, обеспечивающий формирование полюса цепи обратной связи и устойчивость системы.

Таким образом, блок питания обеспечивает стабильное напряжение для цифровой части схемы и высокий ток для силовой части от одного источника.

1.13 Интерфейс программирования

Для загрузки программного обеспечения во Flash-память микроконтроллера предусмотрен разъем XP3. Программирование осуществляется по последовательному интерфейсу ISP (In-System Programming). Разъем подключен к следующим выводам микроконтроллера:

- MOSI (PB5), вход данных для последовательного программирования;
- MISO (PB6), выход данных;

- SCK (PB7), тактовый сигнал интерфейса SPI;
- RESET (Pin 9), для перевода микроконтроллера в режим программирования программатор подает на этот вывод логический ноль;
- VCC и GND, линии питания для согласования уровней сигналов программатора и целевого устройства.

1.14 Расчетная часть

В данном разделе приведены расчеты пассивных компонентов обвязки микросхем, параметров программного обеспечения и энергетического баланса устройства.

1.14.1 Расчет элементов драйвера двигателя

Для защиты электродвигателя от заклинивания микросхема DRV8871 использует функцию ограничения тока I_{TRIP} , порог которой задается резистором на выводе ILIM. Исходя из рабочего тока вентилятора (0.3 А) и необходимости обеспечения запаса для пусковых токов, порог защиты установлен на уровне 2.1 А. Номинал токозадающего резистора R_6 рассчитывается по формуле $R_6[\text{кОм}] = \frac{0.064}{I_{TRIP}}[\text{А}]$, что дает значение 30.5 кОм. На практике выбран ближайший номинал из ряда E24, равный 30 кОм. Управление скоростью вращения вентилятора реализуется методом широтно-импульсной модуляции с использованием восьмибитного таймера микроконтроллера. При тактовой частоте 8 МГц и делителе $N = 8$ частота ШИМ-сигнала составляет $f_{PWM} \approx 3.9$ кГц, что является оптимальным значением для минимизации потерь переключения.

Стабильность работы драйвера обеспечивается внешними конденсаторами. Керамический конденсатор емкостью 100 нФ, установленный в непосредственной близости к выводам питания, подавляет высокочастотные помехи коммутации. Электролитический конденсатор емкостью 22 мкФ служит резервуаром энергии, компенсирующим просадки напряжения в моменты запуска двигателя. Для питания схемы управления верхними ключами H-моста используется вольтодобавочный конденсатор емкостью 100 нФ, подключенный между выводами SW и BS.

1.14.2 Энергетический расчет и выбор источника питания

Расчет общего энергопотребления системы необходим для обоснованного выбора источника питания, способного обеспечить стабильную работу всех компонентов. Энергопотребление устройства складывается из мощности, потребляемой силовой частью (вентилятором), и мощности цифровой части (логики управления).

Цифровая часть схемы питается от напряжения 5 В. Основным потребителем является микроконтроллер ATmega8515, потребляющий в активном режиме при частоте 8 МГц порядка 12 мА, согласно технической документации. Дополнительную нагрузку создают периферийные устройства: жидкокристаллический дисплей с включенной подсветкой (22 мА), цифровой датчик температуры (2.5 мА) и преобразователь интерфейса USB-UART (15 мА). Суммарный ток потребления по линии 5 В составляет 51.5 мА, что эквивалентно мощности 0.26 Вт. Для обеспечения этого питания используется DC-DC преобразователь MP2307. С учетом его КПД порядка 90%, потребление данной части схемы от первичной сети 12 В составит приблизительно 24 мА.

Силовая часть представлена вентилятором с номинальным напряжением 12 В и рабочим током 0.3 А. Потребляемая им мощность составляет 3.6 Вт. Таким образом, суммарный ток, потребляемый всем устройством от источника 12 В в штатном режиме, равен сумме токов силовой и преобразованной логической частей, что составляет 324 мА. Общая потребляемая мощность системы оценивается в 3.9 Вт.

При выборе источника питания необходимо учитывать не только номинальный режим, но и пусковые характеристики электродвигателя, ток которого в момент старта может кратковременно достигать 1.5 А. Исходя из этого, для питания устройства выбран стандартный импульсный блок питания с выходным напряжением 12 В и номинальным током 1 А. Данный источник обеспечивает трехкратный запас по мощности относительно штатного режима потребления, гарантируя надежный запуск вентилятора и отсутствие просадок напряжения, способных вызвать сброс микроконтроллера.

1.14.3 Расчет компонентов периферийных узлов

Для формирования стабильного напряжения 5 В используется понижающий преобразователь, выходное напряжение которого задается резистивным делителем. При нижнем плече делителя 10 кОм расчетное значение верхнего плеча составляет 44.05 кОм. Выбор ближайшего стандартного номинала 43 кОм обеспечивает выходное напряжение 4.9 В, что является безопасным и достаточным для всех цифровых компонентов. Сглаживание выходного напряжения осуществляется LC-фильтром с индуктивностью 10 мкГн и конденсатором 22 мкФ, параметры которых выбраны в соответствии с рекомендациями производителя для минимизации пульсаций.

Настройка интерфейса UART для связи с ПК на скорости 9600 бит/с требует расчета значения регистра делителя частоты UBRR. При тактовой частоте 8 МГц расчетное значение делителя составляет 51. Это обеспечивает реальную скорость обмена 9615 бит/с с погрешностью всего 0.16%, что полностью исключает ошибки синхронизации при передаче данных.

1.14.4 Хранение настроек в энергонезависимой памяти EEPROM

Микроконтроллер ATmega8515 содержит 512 байт энергонезависимой памяти EEPROM, где сохраняются пользовательские настройки при отключении питания. В системе управления охлаждением хранятся пять параметров: режим работы (автоматический или ручной), состояние вентилятора, верхний и нижний пороги температуры, а также скорость вентилятора. Распределение памяти линейное: адрес 0x00 — магический байт инициализации (0xAC), адреса 0x01–0x05 содержат параметры. Всего используется 6 байт из 512, что оставляет 99 процентов емкости в резерве для будущих расширений.

Каждый параметр представляет собой беззнаковое целое число на 8 бит (0–255). Логические значения кодируются как 0 (ложь) и 1 (истина). Пороги температуры хранятся в градусах Цельсия без масштабирования, скорость вентилятора — как коэффициент заполнения ШИМ (0 = выключено, 255 = максимум).

Запись в EEPROM занимает 8.5 миллисекунды за один байт благодаря аппаратным механизмам AVR. Функция `eeprom_update_byte()` оптимизирует процесс, записывая только при изменении значения, что продлевает

жизненный цикл памяти (100 000 циклов записи). При любом изменении параметра через UART-команды или кнопки прошивка немедленно обновляет EEPROM, сохраняя настройки даже при неожиданном отключении питания.

1.14.5 Методы программирования микроконтроллера

Для загрузки программного кода во Flash-память ATmega8515 используется внутрисхемное программирование ISP через разъем XP3. Программатор подключается четырьмя сигналами, MOSI — данные вход, MISO — данные выход, SCK — тактовый сигнал и линией RESET, обеспечивая синхронный последовательный обмен данными с микроконтроллером. Для входа в режим программирования программатор устанавливает RESET на землю, переводя микроконтроллер в специальное состояние, где вместо выполнения пользовательского кода он ожидает команд программирования через интерфейс SPI.

Полный сеанс программирования выполняется за 1–3 секунды и включает следующие этапы: синхронизация и проверка сигнатуры устройства (0x8A для ATmega8515), полное стирание Flash-памяти (20 мс), загрузка нового кода на 128-байтных страницах (каждая записывается 4.5 мс), верификация записанных данных, установка битов конфигурации для выбора источника тактирования и сохранения EEPROM.

Критическое соображение — сохранение содержимого EEPROM при перепрограммировании. По умолчанию операция стирания Flash удаляет и всю EEPROM, теряя пользовательские настройки. Для сохранения настроек между обновлениями прошивки необходимо установить бит конфигурации EESAVE в режим защиты (значение 0). Это стандартная конфигурация для данной системы, позволяющая обновлять код без переконфигурирования устройства.

2 Технологическая часть

2.1 Работа в системах разработки и отладки

Для реализации проекта был выбран комплекс современных программных средств, позволяющий осуществлять сквозное проектирование от разработки алгоритма до прошивки кристалла. Написание и компиляция управляющей программы производились в интегрированной среде AVR Studio. В качестве компилятора использовался AVR-GCC с языком C, что позволило существенно ускорить процесс разработки и повысить читаемость кода по сравнению с использованием языка Ассемблера. Менеджер проектов данной среды предоставляет инструменты для конфигурирования параметров микроконтроллера, таких как тип кристалла, тактовая частота и уровни оптимизации компилятора.

Для программной реализации алгоритмов были использованы внешние библиотеки. Библиотека `avr/io.h` предоставляет определения имен регистров ввода-вывода и битов микроконтроллера ATmega8515, используемых для управления портами и периферией. Библиотека `avr/interrupt.h` предоставляет макрос `ISR` для объявления функций-обработчиков прерываний, а также функции глобального управления прерываниями. Для реализации временных задержек, необходимых для инициализации дисплея и формирования таймингов протокола 1-Wire датчика DHT11, применена библиотека `util/delay.h`.

Особенностью данного проекта является активное использование энергонезависимой памяти и строковых констант. Библиотека `avr/eeprom.h` используется для чтения и сохранения настроек пользователя при отключении питания. Для экономии оперативной памяти, объем которой составляет всего 512 байт, строковые литералы размещены во Flash-памяти программ с помощью макроса `PSTR` из библиотеки `avr/pgmspace.h`. Преобразование строковых команд, поступающих по интерфейсу UART, в числовые значения выполняется функциями стандартной библиотеки `stdlib.h`.

Для проверки аппаратной части без необходимости физической пайки применялся пакет схемотехнического моделирования Proteus 8 Professional.

Ключевой особенностью данной системы является возможность совместной симуляции аналоговых и цифровых компонентов вместе с исполняемым кодом микроконтроллера в реальном времени. В проекте использовались математические модели микроконтроллера ATmega8515, драйвера двигателя DRV8871, символьного дисплея LM016L, датчика DHT11 и виртуального терминала для отладки протокола UART. Симуляция устройства в среде Proteus представлена на рисунке 9

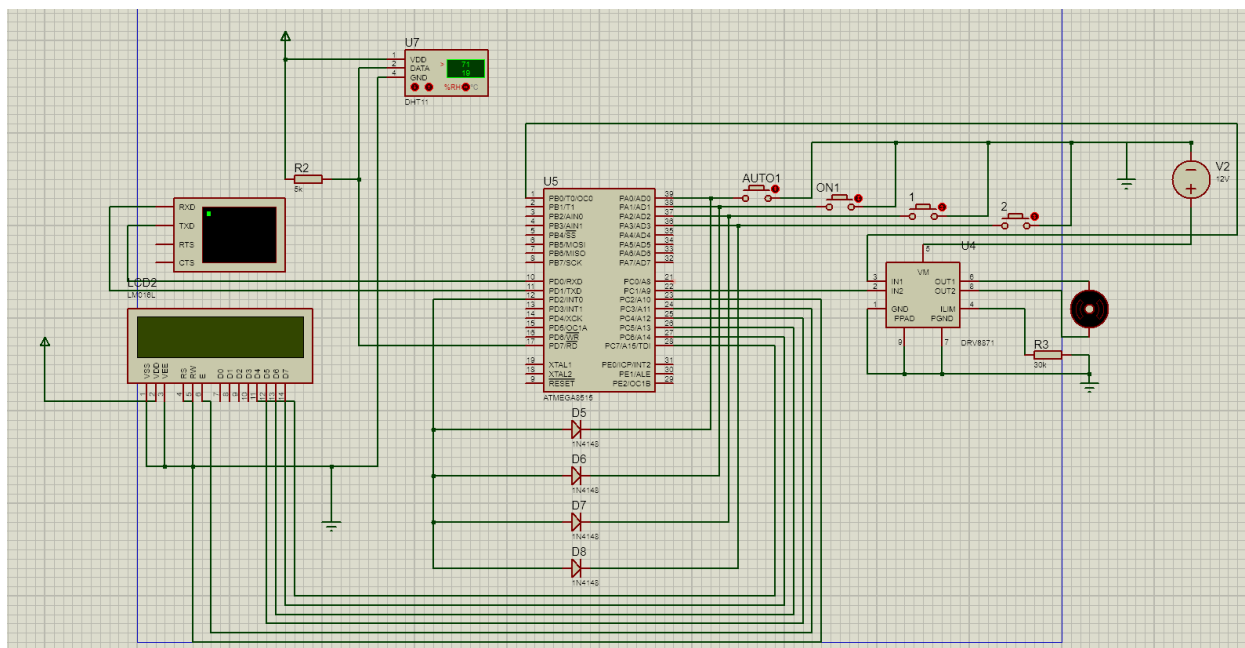


Рисунок 9 — Моделирование в Proteus

2.2 Структура программного обеспечения

Программное обеспечение микроконтроллера разработано на языке высокого уровня C. Архитектура приложения построена по принципу суперцикла с использованием прерываний для обработки асинхронных событий. Программа логически разделена на модуль инициализации, выполняющий настройку периферии при старте, модули драйверов для работы с LCD и датчиком, модуль управления памятью и основной цикл. В основном цикле происходит опрос флагов, реализация автомата состояний термоконтроля и логики управления вентилятором. Обработка критичных ко времени событий, таких как прием данных по UART и реакция на нажатия кнопок, вынесена в обработчики прерываний.

2.3 Настройка периферийных устройств

Конфигурация направлений линий портов выполняется через регистры DDRx. К выходам отнесены линии управления двигателем, линии данных дисплея и ШИМ, а также линия датчика при отправке запроса. Входами являются линии кнопок, для которых программно активированы внутренние подтягивающие резисторы для обеспечения стабильного логического уровня в ненажатом состоянии.

Для управления скоростью вращения вентилятора используется 8-битный таймер-счетчик T0 в режиме быстрого ШИМ. Настройка производится через регистр TCCR0 путем установки битов, включающих режим Fast PWM, неинвертирующий режим выхода на выводе OC0 и делитель тактовой частоты. Это обеспечивает генерацию сигнала с частотой, оптимальной для управления драйвером двигателя.

2.4 Отладка и тестирование

Для подтверждения работоспособности разработанного программного обеспечения проведено тестирование функциональных модулей системы. Тестирование включало проверку реакции устройства на нажатие кнопок управления, прием команд по интерфейсу UART и автоматическую работу системы термоконтроля. Проверки выполнялись в среде симуляции Proteus.

Результаты тестирования приведены в таблице 6.

Таблица 6 — Таблица тестовых случаев

Входные данные	Ожидаемый результат	Фактический результат
Нажатие кнопки «AUTO» (PA0)	Переход в автоматический режим. Отправка «INT: AUTO Mode» по UART	Система перешла в режим AUTO. Сообщение получено
Нажатие кнопки «TOGGLE» (PA1)	Переключение в ручной режим. Смена состояния мотора. Отправка «INT: Toggle Man»	Режим изменен на MANUAL. Состояние мотора изменилось

Продолжение таблицы 6

Нажатие кнопки «UP» (PA2)	Увеличение скорости на 25. Обновление OCR0. Отправка «INT: Speed UP: «	Скорость увеличилась. ШИМ изменился
Команда «A» по UART	Переход в автоматический режим. Ответ «Mode: AUTO»	Режим AUTO. Ответ корректен
Команда «1» по UART	Включение мотора в ручном режиме. Ответ «Manual ON»	Мотор включен. Ответ получен
Команда «V150» по UART	Установка скорости 150. Обновление ШИМ. Сохранение в EEPROM	Скорость установлена 150. ШИМ соответствует
Команда «H30» по UART	Установка верхнего порога 30°C. Ответ «High Saved: 30»	Порог изменен. Ответ получен
Режим AUTO. Температура 24°C, Порог H=23°C	Включение вентилятора	Вентилятор включился
Режим AUTO. Температура 20°C, Порог L=21°C	Выключение вентилятора	Вентилятор выключился

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было спроектировано, программно реализовано и протестировано микропроцессорное устройство для автоматизированного управления системой охлаждения. Поставленная цель работы достигнута, а требования технического задания выполнены в полном объеме.

В процессе проектирования получены следующие основные результаты:

1) обоснован выбор элементной базы и разработана схема электрическая принципиальная, обеспечивающая корректное согласование микроконтроллера ATmega8515 с цифровым датчиком DHT11 и силовым драйвером DRV8871;

2) написана программа управления на языке C, содержащая драйверы периферии, модуль для работы с нестандартным 1-wire протоколом и алгоритмы автоматического регулирования;

3) организован развитый пользовательский интерфейс, сочетающий локальную индикацию параметров на символьном ЖК-дисплее и возможность удаленной настройки через последовательный интерфейс UART;

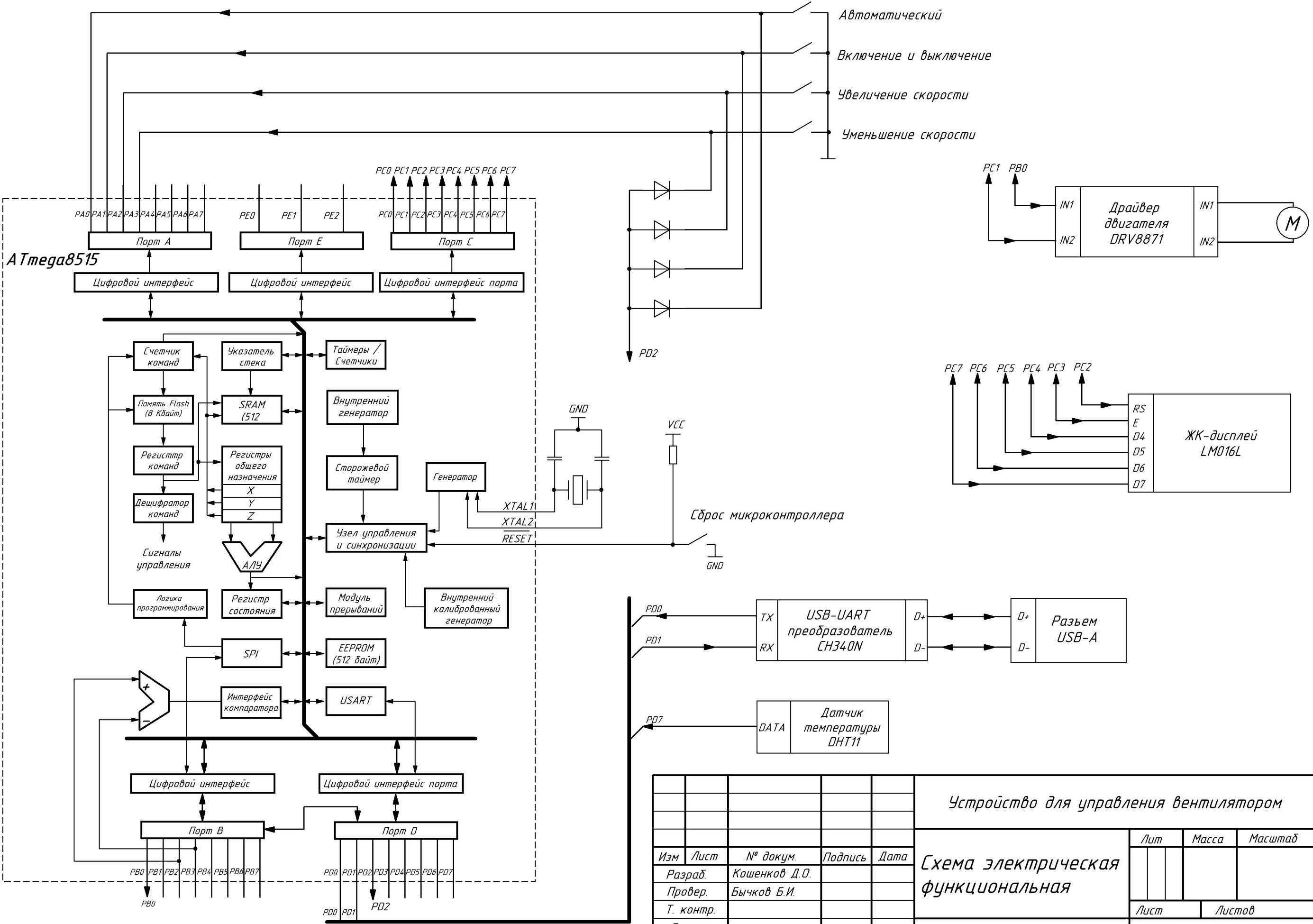
4) внедрена подсистема работы с энергонезависимой памятью (EEPROM), обеспечивающая сохранение уставок температуры и режима работы при отключении питания;

5) проведено тестирование и отладка устройства, подтвердившие корректность формирования временных интервалов и стабильность работы во всех режимах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ATmega8515 8-bit Microcontroller with 8K Bytes In-System Programmable Flash. Datasheet [Электронный ресурс]. URL: <https://ww1.microchip.com/downloads/en/DeviceDoc/doc2512.pdf> (дата обращения: 20.05.2025).
2. DHT11 Humidity & Temperature Sensor. Datasheet [Электронный ресурс]. URL: <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf> (дата обращения: 20.05.2025).
3. DRV8871 3.6-V to 40-V Brushed DC Motor Driver with Internal Current Sensing. Datasheet [Электронный ресурс]. URL: <https://www.ti.com/lit/ds/symlink/drv8871.pdf> (дата обращения: 20.05.2025).
4. LM016L Liquid Crystal Display Module. Datasheet [Электронный ресурс]. URL: <https://www.hitachi-displays-eu.com/doc/LM016L.pdf> (дата обращения: 20.05.2025).
5. CH340 USB to Serial Chip. Datasheet [Электронный ресурс]. URL: https://www.wch-ic.com/downloads/CH340DS1_PDF.html (дата обращения: 20.05.2025).
6. MP2307 3A, 23V, 340KHz Synchronous Rectified Step-Down Converter. Datasheet [Электронный ресурс]. URL: https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Datasheet/lang/en/sku/MP2307/ (дата обращения: 20.05.2025).
7. ГОСТ 2.702-2011. Единая система конструкторской документации. Правила выполнения электрических схем. 2011.
8. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. 1990.
9. AVR Libc Reference Manual [Электронный ресурс]. URL: <https://www.nongnu.org/avr-libc/user-manual/> (дата обращения: 20.05.2025).

ПРИЛОЖЕНИЕ А
ФУНКЦИОНАЛЬНАЯ ЭЛЕКТРИЧЕСКАЯ СХЕМА
Листов 1



					Устройство для управления вентилятором				
Изм	Лист	№ докум.	Подпись	Дата	Схема электрическая функциональная	Лит		Масса	Масштаб
Разраб.		Кошенков Д.О.							
Провер.		Бычков Б.И.							
Т. контр.									
Реценз.									
Н. контр.						Лист		Листов	
Утверд.						МГТУ им. Н.Э. Баумана Группа ИУ6-74Б			

ПРИЛОЖЕНИЕ Б
ФУНКЦИОНАЛЬНАЯ ЭЛЕКТРИЧЕСКАЯ СХЕМА
Листов 1

Перв. примен.

Справ. №

Подп. и дата

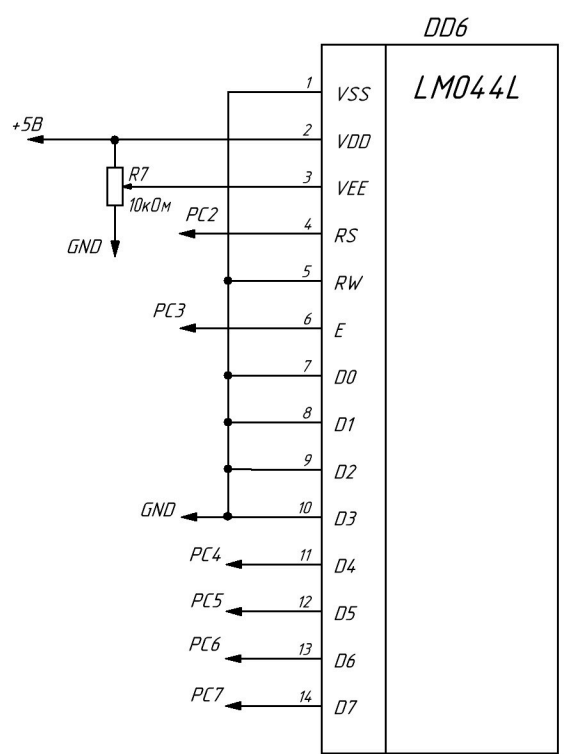
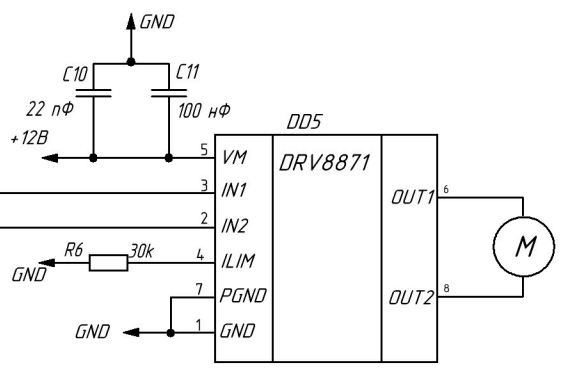
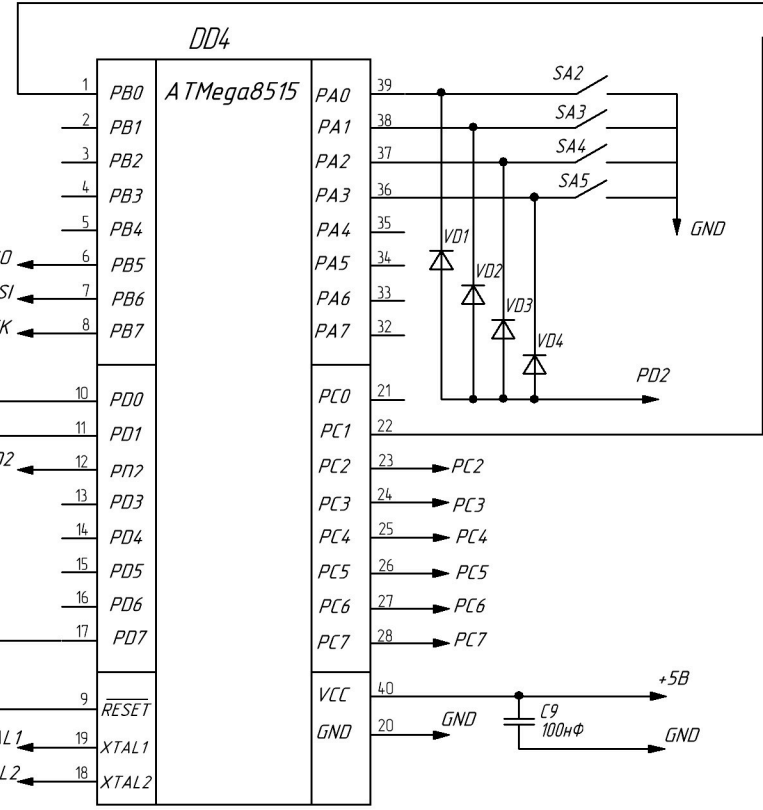
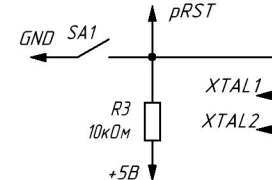
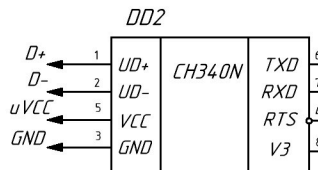
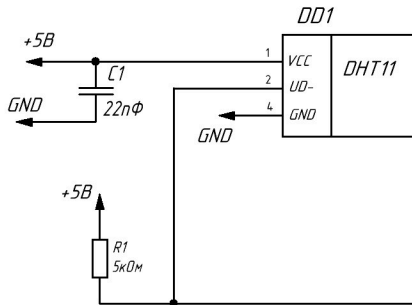
Инв. № дудл.

Взам. инв. №

Подп. и дата

Инв. № подл.

Модуль управления
вентилятором



XP1

Назначение	Цель	Контакт
Питание	VCC	1
Земля	GND	2

Разъем питания

XP2

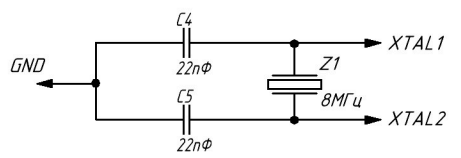
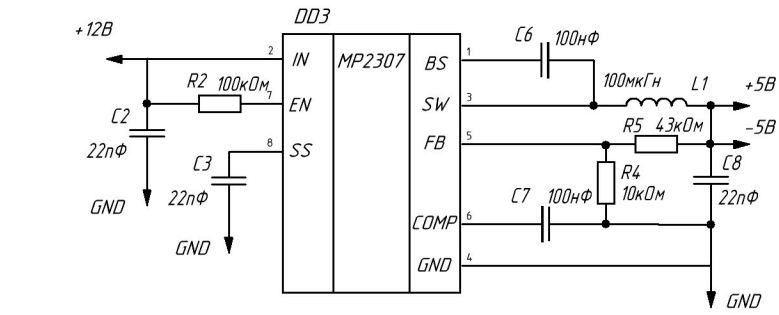
Назначение	Цель	Контакт
Питание	uVCC	1
Линия данных	D+	2
Линия данных	D-	3
Земля	uGND	4

Разъем USB-A

XP3

Назначение	Цель	Контакт
Линия данных	pMISO	1
Питание	pVCC	2
Тактовый сигнал	pSCK	3
Линия данных	pMOSI	4
Сброс	pRST	5
Земля	pGND	6

Разъем программатора



					Устройство для управления вентилятором					
					Схема электрическая функциональная	Лист			Масса	Масштаб
Изм	Лист	№ докум.	Подпись	Дата						
Разраб.		Кошенков Д.О.								
Провер.		Бычков Б.И.								
Т. контр.										
Реценз.						Лист			Листов 1	
Н. контр.						МГТУ им. Н.Э. Баумана				
Утверд.						Группа ИУ6-74Б				

Поз. обознач.		Наименование				Кол.	Примечание		
		<u>Схемы интегральные</u>							
DD1		DHT11				1			
DD2		CH340N				1			
DD3		MP2307				1			
DD4		ATMEGA8515				1			
DD5		DRV8871				1			
DD5		LM044M				1			
		<u>Конденсаторы электрические</u>							
C1-C5,C10		Керамический К10-17Б, 22нФ				6			
C6-C9,C11		Керамический К10-17Б, 100нФ				5			
		<u>Резисторы</u>							
R1		МЛТ 0,125 - 5кОм				1			
R2-R5,R7		МЛТ 0,125 - 10кОм				5			
R6		МЛТ 0,125 - 30кОм				1			
		<u>Кнопки</u>							
SA1-SA5		KLS7-TS3601				5			
		<u>Диоды</u>							
VD1-VD4		1N4933				1			
		<u>Индукторы</u>							
L1		SMD CDRH104R-100NC 10 мкГн				1			

[illegible]

ПРИЛОЖЕНИЕ В
ИСХОДНЫЙ ТЕКСТ ПРОГРАММЫ
Листов 10

Листинг В.1 — Содержимое файла main.c

```
#define F_CPU 8000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/eeprom.h>
#include <avr/pgmspace.h>
#include <util/delay.h>
#include <stdlib.h>
#include <string.h>

// --- НАСТРОЙКИ ПИНОВ ---

// Моторы (PC0, PC1)
#define MOTOR_PORT PORTC
#define MOTOR_DDR DDRC
#define IN1_PIN PC0
#define IN2_PIN PC1

// ШИМ (PB0)
#define PWM_PORT PORTB
#define PWM_DDR DDRB
#define PWM_PIN PB0

// Датчик DHT (PD7)
#define DHT_PORT PORTD
#define DHT_DDR DDRD
#define DHT_PIN PD7
#define DHT_PIN_IN PIND

// КНОПКИ (PORTA)
#define BTN_PORT PORTA
#define BTN_PIN PINA
#define BTN_DDR DDRA
#define BTN_AUTO PA0
#define BTN_TOGGLE PA1
#define BTN_UP PA2
#define BTN_DOWN PA3

#define INT0_PIN PD2

// LCD ЭКРАН (PC2-PC7)
#define LCD_PORT PORTC
#define LCD_DDR DDRC
#define LCD_RS PC2
#define LCD_E PC3
```

Продолжение листинга В.1

```
// eeprom
#define EE_MAGIC_VAL 0xAC

uint8_t EEMEM ee_magic;
uint8_t EEMEM ee_mode_auto;
uint8_t EEMEM ee_fan_on;
uint8_t EEMEM ee_temp_high;
uint8_t EEMEM ee_temp_low;
uint8_t EEMEM ee_fan_speed;

// переменные
volatile uint8_t fan_on = 0;
volatile uint8_t mode_auto = 1;
volatile uint8_t temp_c = 0;
volatile uint8_t humidity = 0;

volatile uint8_t temp_high = 23;
volatile uint8_t temp_low = 21;
volatile uint8_t fan_speed = 255;

// Буфер UART
volatile char rx_buffer[20];
volatile uint8_t rx_pos = 0;
volatile uint8_t cmd_ready = 0;

volatile uint8_t update_lcd_needed = 1;

// --- ФУНКЦИИ LCD ---
void lcd_pulse(void) {
    LCD_PORT |= (1 << LCD_E);
    _delay_us(5);
    LCD_PORT &= ~(1 << LCD_E);
    _delay_us(50);
}

void lcd_byte(uint8_t val, uint8_t is_data) {
    if (is_data) LCD_PORT |= (1 << LCD_RS);
    else        LCD_PORT &= ~(1 << LCD_RS);

    uint8_t temp = (LCD_PORT & 0x0F);
    temp |= (val & 0xF0);
    LCD_PORT = temp;
    lcd_pulse();

    temp = (LCD_PORT & 0x0F);
    temp |= ((val << 4) & 0xF0);
}
```

Продолжение листинга В.1

```
        LCD_PORT = temp;
        lcd_pulse();
    }

void lcd_cmd(uint8_t cmd) {
    lcd_byte(cmd, 0);
    if (cmd == 0x01 || cmd == 0x02) _delay_ms(2);
}

void lcd_char(char data) { lcd_byte(data, 1); }

void lcd_str_P(const char *str) {
    while (pgm_read_byte(str)) lcd_char(pgm_read_byte(str++));
}

void lcd_str(const char *str) { while (*str) lcd_char(*str++); }

void lcd_num(int val) {
    char buf[10];
    itoa(val, buf, 10);
    lcd_str(buf);
}

void lcd_init(void) {
    LCD_DDR |= (1 << LCD_RS) | (1 << LCD_E) | 0xF0;
    _delay_ms(20);
    LCD_PORT &= ~(1 << LCD_RS);

    LCD_PORT = (LCD_PORT & 0x0F) | 0x30; lcd_pulse();
    _delay_ms(5);
    LCD_PORT = (LCD_PORT & 0x0F) | 0x30; lcd_pulse();
    _delay_us(200);
    LCD_PORT = (LCD_PORT & 0x0F) | 0x30; lcd_pulse();
    _delay_us(200);
    LCD_PORT = (LCD_PORT & 0x0F) | 0x20; lcd_pulse();
    _delay_ms(5);

    lcd_cmd(0x28);
    lcd_cmd(0x0C);
    lcd_cmd(0x06);
    lcd_cmd(0x01);
}

void lcd_gotoxy(uint8_t x, uint8_t y) {
    uint8_t addr = (y == 0) ? 0x80 : 0xC0;
```

Продолжение листинга В.1

```
        lcd_cmd(addr + x);
    }
void load_config(void) {
    uint8_t magic = eeprom_read_byte(&ee_magic);
    if (magic != EE_MAGIC_VAL) {
        eeprom_update_byte(&ee_mode_auto, 1);
        eeprom_update_byte(&ee_fan_on, 0);
        eeprom_update_byte(&ee_temp_high, 23);
        eeprom_update_byte(&ee_temp_low, 21);
        eeprom_update_byte(&ee_fan_speed, 255);
        eeprom_update_byte(&ee_magic, EE_MAGIC_VAL);
    }
    mode_auto = eeprom_read_byte(&ee_mode_auto);
    fan_on = eeprom_read_byte(&ee_fan_on);
    temp_high = eeprom_read_byte(&ee_temp_high);
    temp_low = eeprom_read_byte(&ee_temp_low);
    fan_speed = eeprom_read_byte(&ee_fan_speed);
}

// --- UART ---

void uart_init_fixed(void) {
    UBRRH = 0; UBRRL = 51; // 9600
    UCSRB = (1 << RXCIE) | (1 << RXEN) | (1 << TXEN);
    UCSRC = (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0);
}

void uart_send_char(char c) { while (!(UCSRA & (1 << UDRE)));
    UDR = c; }
void uart_send_str(const char *s) { while (*s)
    uart_send_char(*s++); }
void uart_send_str_P(const char *s) {
    while (pgm_read_byte(s)) uart_send_char(pgm_read_byte(s+
+));
}
void uart_send_num(int val) {
    char buf[10];
    itoa(val, buf, 10);
    uart_send_str(buf);
}

ISR(USART_RX_vect) {
    char data = UDR;
    if (data == '\r' || data == '\n') {
        rx_buffer[rx_pos] = 0; cmd_ready = 1; rx_pos = 0;
    } else {
```


Продолжение листинга В.1

```
        if (rx_pos < 18) rx_buffer[rx_pos++] = data;
    }
}

// --- УПРАВЛЕНИЕ МОТОРОМ ---

void set_motor(uint8_t state) {
    fan_on = state;
    eeprom_update_byte(&ee_fan_on, state);
    if (state) {
        MOTOR_PORT |= (1 << IN1_PIN);
        MOTOR_PORT &= ~(1 << IN2_PIN);
        OCR0 = fan_speed;
    } else {
        MOTOR_PORT &= ~((1 << IN1_PIN) | (1 << IN2_PIN));
        OCR0 = 0;
    }
    update_lcd_needed = 1;
}

// --- ОБРАБОТЧИК ПРЕРЫВАНИЯ КНОПОК INT0 ---

void init_int0(void) {
    DDRD &= ~(1 << INT0_PIN);
    PORTD |= (1 << INT0_PIN);

    MCUCR |= (1 << ISC01);
    MCUCR &= ~(1 << ISC00);

    GICR |= (1 << INT0);
}

ISR(INT0_vect) {
    _delay_ms(30);

    // Кнопка AUTO
    if (!(BTN_PIN & (1 << BTN_AUTO))) {
        mode_auto = 1;
        eeprom_update_byte(&ee_mode_auto, 1);
        uart_send_str_P(PSTR("INT: AUTO Mode\r\n"));
        update_lcd_needed = 1;
    }
    // Кнопка TOGGLE (Manual ON/OFF)
    else if (!(BTN_PIN & (1 << BTN_TOGGLE))) {
        mode_auto = 0;
        eeprom_update_byte(&ee_mode_auto, 0);
    }
}
```

Продолжение листинга В.1

```
        set_motor(!fan_on);
        uart_send_str_P(PSTR("INT: Toggle Man\r\n"));
        update_lcd_needed = 1;
    }
    // Кнопка UP
    else if (!(BTN_PIN & (1 << BTN_UP))) {
        int new_speed = fan_speed + 25;
        if (new_speed >= 255) new_speed = 255;
        fan_speed = (uint8_t)new_speed;
        eeprom_update_byte(&ee_fan_speed, fan_speed);
        if (fan_on) OCR0 = fan_speed;

        uart_send_str_P(PSTR("INT: Speed UP: "));
        uart_send_num(fan_speed); uart_send_str_P(PSTR("\r\n"));
        update_lcd_needed = 1;
    }
    // Кнопка DOWN
    else if (!(BTN_PIN & (1 << BTN_DOWN))) {
        int new_speed = fan_speed - 25;
        if (new_speed <= 0) new_speed = 0;
        fan_speed = (uint8_t)new_speed;
        eeprom_update_byte(&ee_fan_speed, fan_speed);
        if (fan_on) OCR0 = fan_speed;

        uart_send_str_P(PSTR("INT: Speed DOWN: "));
        uart_send_num(fan_speed); uart_send_str_P(PSTR("\r\n"));
        update_lcd_needed = 1;
    }

    while(!(PIND & (1 << INT0_PIN)));
}

// --- DHT ДАТЧИК ---

uint8_t dht_read(void) {
    uint8_t bits[5]; uint8_t cnt = 7; uint8_t idx = 0;
    for(int k=0; k<5; k++) bits[k] = 0;

    DHT_DDR |= (1 << DHT_PIN); DHT_PORT &= ~(1 << DHT_PIN);
    _delay_ms(20); DHT_PORT |= (1 << DHT_PIN); DHT_DDR &= ~(1
<< DHT_PIN);

    _delay_us(40); if ((DHT_PIN_IN & (1 << DHT_PIN))) return
1;
    _delay_us(80); if (!(DHT_PIN_IN & (1 << DHT_PIN))) return
2;
```

Продолжение листинга В.1

```
    _delay_us(80);

    for (int i = 0; i < 40; i++) {
        uint8_t timeout = 0;
        while(!(DHT_PIN_IN & (1 << DHT_PIN))) { _delay_us(1);
if (++timeout > 100) return 3; }
        _delay_us(30);
        if (DHT_PIN_IN & (1 << DHT_PIN)) {
            if (idx < 5) bits[idx] |= (1 << cnt);
            timeout = 0;
            while(DHT_PIN_IN & (1 << DHT_PIN)) { _delay_us(1);
if (++timeout > 100) return 4; }
        }
        if (cnt == 0) { cnt = 7; idx++; } else { cnt--; }
    }
    if ((uint8_t)(bits[0] + bits[1] + bits[2] + bits[3]) ==
bits[4]) {
        humidity = bits[0]; temp_c = bits[2]; return 0;
    }
    return 5;
}

void update_lcd_info(void) {
    lcd_gotoxy(0, 0);
    lcd_str_P(PSTR("T:")); lcd_num(temp_c); lcd_char(0xDF);
    lcd_str_P(PSTR("C H:")); lcd_num(humidity);
    lcd_str_P(PSTR("% ")); lcd_char(mode_auto ? 'A' : 'M');

    lcd_gotoxy(0, 1);
    if (fan_on) {
        lcd_str_P(PSTR("ON Spd:")); lcd_num(fan_speed);
    lcd_str_P(PSTR("    "));
    } else {
        lcd_str_P(PSTR("OFF H:")); lcd_num(temp_high);
    lcd_str_P(PSTR(" L:")); lcd_num(temp_low);
    }
}

int main(void) {
    MOTOR_DDR |= (1 << IN1_PIN) | (1 << IN2_PIN);
    PWM_DDR |= (1 << PWM_PIN);

    TCCR0 = (1 << WGM00) | (1 << WGM01) | (1 << COM01) | (1 <<
CS01);
    OCR0 = 0;
```

Продолжение листинга В.1

```
    BTN_DDR &= ~(1 << BTN_AUTO) | (1 << BTN_TOGGLE) | (1 <<
BTN_UP) | (1 << BTN_DOWN));
    BTN_PORT |= (1 << BTN_AUTO) | (1 << BTN_TOGGLE) | (1 <<
BTN_UP) | (1 << BTN_DOWN);

    uart_init_fixed();
    load_config();
    lcd_init();

    set_motor(fan_on);

    init_int0();

    sei();

    uart_send_str_P(PSTR("System Ready (INT0 Mode)\r\n"));
    lcd_cmd(0x01); lcd_gotoxy(0, 0); lcd_str_P(PSTR("System
Ready"));
    _delay_ms(1000); lcd_cmd(0x01);

    uint8_t timer_ticks = 0;

    while (1) {
        if (cmd_ready) {
            _delay_ms(5);
            char *cmd = (char*)rx_buffer;

            if (cmd[0] == '1') {
                mode_auto = 0;
                eeprom_update_byte(&ee_mode_auto, 0);
                set_motor(1);
                uart_send_str_P(PSTR("Manual ON\r\n"));
            }
            else if (cmd[0] == '0') {
                mode_auto = 0;
                eeprom_update_byte(&ee_mode_auto, 0);
                set_motor(0);
                uart_send_str_P(PSTR("Manual OFF\r\n"));
            }
            else if (cmd[0] == 'A' || cmd[0] == 'a') {
                mode_auto = 1;
                eeprom_update_byte(&ee_mode_auto, 1);
                uart_send_str_P(PSTR("Mode: AUTO\r\n"));
                update_lcd_needed = 1;
            }
            else if (cmd[0] == 'M' || cmd[0] == 'm') {
```

Продолжение листинга В.1

```
        mode_auto = 0;
        eeprom_update_byte(&ee_mode_auto, 0);
        uart_send_str_P(PSTR("Mode: MANUAL\r\n"));
        update_lcd_needed = 1;
    }
    else if (cmd[0] == 'V' || cmd[0] == 'v') {
        int val = atoi(&cmd[1]);
        if (val >= 0 && val <= 255) {
            fan_speed = val;
            eeprom_update_byte(&ee_fan_speed, val);
            if(fan_on) OCR0 = fan_speed;
            update_lcd_needed = 1;
        }
    }
    else if (cmd[0] == 'H' || cmd[0] == 'h') {
        int val = atoi(&cmd[1]);
        if (val > temp_low && val < 99) {
            temp_high = val;
            eeprom_update_byte(&ee_temp_high, val);
            update_lcd_needed = 1;
        }
    }
    else if (cmd[0] == 'L' || cmd[0] == 'l') {
        int val = atoi(&cmd[1]);
        if (val > 0 && val < temp_high) {
            temp_low = val;
            eeprom_update_byte(&ee_temp_low, val);
            update_lcd_needed = 1;
        }
    }
    cmd_ready = 0;
}

_delay_ms(100);
timer_ticks++;
if (timer_ticks >= 20) {
    timer_ticks = 0;
    if (dht_read() == 0) {
        update_lcd_needed = 1;
        if (mode_auto) {
            if (temp_c >= temp_high && !fan_on)
                set_motor(1);
            else if (temp_c <= temp_low && fan_on)
                set_motor(0);
        }
    }
}
```

Продолжение листинга В.1

```
        }  
  
        if (update_lcd_needed) {  
            update_lcd_info();  
            update_lcd_needed = 0;  
        }  
    }  
}
```